

Number: P3378R0
Date: 2024-10-14
Audience: LEWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Introduction and motivation](#)
- [TL;DR](#)
- [Implementation experience](#)
 - [libc++](#)
 - [Reference counted string](#)
 - [libc++abi](#)
 - [Dependency on <string>](#)
- [Existing exception types](#)
- [Should all exception types be constexpr?](#)
 - [std::runtime_error](#)
 - [std::error_code and std::error_category depending exception types](#)
- [Impact on existing code](#)
- [Intention for wording changes](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

constexpr exception types

This paper is mechanical wording change; making all exception types associated with constexpr compatible functionality marked constexpr. This is needed for consistency across library since allowing exception handling during constant evaluation by [P3068](#).

As every exception type is just an ordinary type, they should be make constexpr compatible.

TL;DR

Now when we can throw exceptions during constant evaluation we should make sure all constexpr compatible functionality (eg. `vector`) is able to throw its exception types (eg. `out_of_range`) and allow users to recover from errors gracefully.

This proposal fixes the lag between constexpr exception support and library constexpr support and all future constexprification papers should make their exception types constexpr.

Implementation experience

Implemented in libc++ as part of implementation of [P3068](#), with source code available on [my github](#) and with compiler + library available on [the compiler explorer](#).

```
constexpr bool check(const char * msg) {  
    try {  
        auto vec = std::vector<int>{0,1,2,3,4};  
        return vec.at(4) == 4; // out-of-range  
    } catch (const std::out_of_range & e) {  
        return std::string_view{exc.what()};  
    } catch (...) {  
        return false;  
    }  
    return false;  
}
```